

Entity States in Entity Framework Core

Entity Framework Core tracks entities in different states to manage database operations. Here are the 5 main states:

1. ****Added**** (New Entity)

```
```csharp
// Your Employee model
var newEmployee = new NETEmployee
{
 FirstName = "John",
 LastName = "Doe",
 Email = "john@example.com",
 Salary = 50000
};

_context.Employees.Add(newEmployee);
// State: Added
// EF will generate INSERT statement when SaveChanges() is called
await _context.SaveChangesAsync();
```
```

2. ****Unchanged**** (No modifications)

```
```csharp
// Entity loaded from database
var employee = await _context.Employees.FindAsync(1);
// State: Unchanged
// EF knows this entity matches database state
// No SQL will be generated for this entity on SaveChanges()
```
```

3. ****Modified**** (Entity has changes)

```
```csharp
var employee = await _context.Employees.FindAsync(1);
employee.Salary = 60000; // Change property
// State: Modified
// EF will generate UPDATE statement for this entity
```

```
await _context.SaveChangesAsync();
` ` `
```

### 4. **\*\*Deleted\*\*** (Marked for deletion)

```
` ` ` csharp
var employee = await _context.Employees.FindAsync(1);
_context.Employees.Remove(employee);
// State: Deleted
// EF will generate DELETE statement when SaveChanges() is called
await _context.SaveChangesAsync();
` ` `
```

### 5. **\*\*Detached\*\*** (Not tracked)

```
` ` ` csharp
var employee = new NETEmployee
{
 Eid = 1,
 FirstName = "John"
};
// State: Detached
// EF doesn't know about this entity
// No database operations will occur
` ` `
```

## Entity State Lifecycle with Your Employee Model

```
` ` ` csharp
public async Task<IActionResult> UpdateEmployee(NETEmployee employee)
{
 // 1. Create new entity (Detached state)
 var updatedEmployee = new NETEmployee
 {
 Eid = employee.Eid,
 FirstName = employee.FirstName,
 LastName = employee.LastName,
 Salary = employee.Salary
 };
 // State: Detached
}
```

```

// 2. Attach to context (Unchanged state)
_context.Employees.Attach(updatedEmployee);
// State: Unchanged

// 3. Mark as modified (Modified state)
_context.Entry(updatedEmployee).State = EntityState.Modified;
// State: Modified

await _context.SaveChangesAsync();
// EF generates UPDATE statement
}
...

AsNoTracking() Method

What AsNoTracking() Does
`AsNoTracking()` tells EF Core not to track changes to entities. This improves
performance for read-only operations.

Without AsNoTracking() (Tracking)
```csharp
// Your current repository method
public async Task<IEnumerable<NETEmployee>> GetAllEmployeesAsync()
{
    var employees = await _context.Employees.ToListAsync();
    // State: Unchanged for all employees
    // EF tracks all entities - uses memory

    // If you modify any employee:
    employees.First().Salary = 70000; // EF detects this change
    await _context.SaveChangesAsync(); // Generates UPDATE statement
}
...

### With AsNoTracking() (No Tracking)
```csharp
public async Task<IEnumerable<NETEmployee>> GetAllEmployeesAsync()

```

```
{
 var employees = await _context.Employees
 .AsNoTracking() // <-- Key difference
 .ToListAsync();
 // State: Detached for all employees
 // EF doesn't track entities - saves memory

 // If you modify any employee:
 employees.First().Salary = 70000; // EF ignores this change
 await _context.SaveChangesAsync(); // No UPDATE statement generated
}
` ``
```

## ## Performance Comparison

### ### Tracking Query (Default)

```
` `` csharp
// Uses more memory, slower for large datasets
var employees = await _context.Employees
 .Include(e => e.Department) // Tracks Department too
 .ToListAsync();
// Memory: ~100KB for 1000 employees
// Time: ~50ms
` ``
```

### ### NoTracking Query

```
` `` csharp
// Uses less memory, faster for read-only operations
var employees = await _context.Employees
 .AsNoTracking()
 .Include(e => e.Department)
 .ToListAsync();
// Memory: ~60KB for 1000 employees
// Time: ~30ms
` ``
```

## ## When to Use AsNoTracking()

### ✅ Use AsNoTracking() for:

- \*\*Read-only API endpoints\*\* (GET operations)
- \*\*Large data exports\*\*
- \*\*Reporting queries\*\*
- \*\*Display data in UI\*\*

### ❌ Don't use AsNoTracking() for:

- \*\*Update operations\*\* (PUT/PATCH)
- \*\*Delete operations\*\*
- \*\*When you need to modify entities\*\*

## ## Updated Repository Example

```
```csharp
public class EmployeeRepository : IEmployeeRepository
{
    // Read-only operation - use AsNoTracking
    public async Task<IEnumerable<NETEmployee>> GetAllEmployeesAsync()
    {
        return await _context.Employees
            .AsNoTracking()
            .Include(e => e.Department)
            .ToListAsync();
    }

    // Read-only with pagination - use AsNoTracking
    public async Task<PagedResult<NETEmployee>>
    GetAllEmployeesPagination(PaginationParams params)
    {
        var query = _context.Employees
            .AsNoTracking()
            .Include(e => e.Department);

        return await query
            .Skip((params.PageNumber - 1) * params.PageSize)
            .Take(params.PageSize)
            .ToListAsync();
    }
}
```

```

    }

    // Update operation - DON'T use AsNoTracking
    public async Task<NETEmployee> UpdateEmployee(NETEmployee employee)
    {
        _context.Employees.Update(employee); // Requires tracking
        await _context.SaveChangesAsync();
        return employee;
    }
}
```

```

## ## Entity State Monitoring

You can check entity states:

```

```csharp
var employee = await _context.Employees.FindAsync(1);
Console.WriteLine(_context.Entry(employee).State); // "Unchanged"

employee.Salary = 60000;
Console.WriteLine(_context.Entry(employee).State); // "Modified"
```

```

This explains how EF Core manages entity lifecycles and why `AsNoTracking()` is important for performance in read-only scenarios.

## ## Summary

I've explained Entity Framework Core's entity states and `AsNoTracking()` :

### **\*\*Entity States:\*\***

- **\*\*Added\*\*** - New entities (INSERT)
- **\*\*Unchanged\*\*** - No changes (no SQL)
- **\*\*Modified\*\*** - Changed entities (UPDATE)
- **\*\*Deleted\*\*** - Marked for deletion (DELETE)
- **\*\*Detached\*\*** - Not tracked (no SQL)

**\*\*AsNoTracking():\*\***

- **\*\*Improves performance\*\*** for read-only operations
- **\*\*Saves memory\*\*** by not tracking entities
- **\*\*Prevents accidental updates\*\*** in GET operations
- **\*\*Use for\*\***: GET endpoints, pagination, reporting
- **\*\*Don't use for\*\***: PUT/PATCH/DELETE operations

Your current repository methods would benefit from ``AsNoTracking()`` for read operations while keeping tracking for update operations.